

Truck Asset Matchmaking Platform (TAMP)

Solution Overview

1. Introduction

The Truck Asset Matchmaking Platform (TAMP) is a web-based freight marketplace designed to connect Freight Owners who need to transport cargo with Transporters that have available truck capacity. The platform simplifies the process of finding suitable transporters, improves truck utilisation, and provides a transparent workflow from load creation to delivery tracking.

This project represents a simplified Minimum Viable Product (MVP) developed using mock data and frontend technologies to demonstrate the complete business workflow described in the TAMP Business Requirements Specification (BRS).

2. Project Scope

The scope of this MVP is limited to the frontend implementation of the TAMP platform using React and mock data.

The application demonstrates the complete user journey for three different user roles:

- Freight Owner
- Transporter
- Administrator

The solution includes role-based navigation, cargo and truck posting, rule-based matchmaking, digital confirmation, trip tracking, and administrative management features.

The application does not include a production backend, database, authentication server, or third-party integrations.

3. Technology Stack

The application was developed using the following technologies:

Technology	Purpose
React	User Interface
React Router DOM	Development Environment
Tailwind css	Client-side routing
Luccide React	Styling and responsive design
Javascript (ES6)	Icons

Local State & Mock Data	Simulated Backend
-------------------------	-------------------

4. Major Components

The application is organised into several functional modules.

Authentication

- Login
- Register
- Forgot Password
- Role Selection

Freight Owner

- Dashboard
- Post Load
- My Loads
- Match Recommendations
- Digital Confirmation
- Trip Tracking

Transporter

- Dashboard
- Post Truck
- My Trucks
- Available Loads
- Trip Tracking

Administrator

- Dashboard
- Users Management
- Compliance Management
- Audit Trail
- Analytics

Shared Components

- Navbar
- Sidebar
- Layout
- Dashboard Cards
- Buttons
- Forms

5. Key Design Decisions

Several design decisions were made during the development of this MVP.

- **Component-Based Architecture**
React components were used throughout the project to encourage code reuse, improve maintainability, and reduce duplication.
- **Role-Based Navigation**
The application displays different navigation menus depending on whether the user is a Freight Owner, Transporter, or Administrator.
- **Mock Data**
- **Responsive Design**
Tailwind CSS was used to ensure that pages remain usable across desktop and tablet screen sizes.
- **Consistent User Experience**
A shared Layout component provides consistent navigation, spacing, typography, and styling throughout the application.
- **Rule-Based Matchmaking**
Rather than implementing Artificial Intelligence, transporter recommendations are generated using simplified business rules based on:
 - Vehicle compatibility
 - Capacity
 - Availability
 - Location
 - Transporter rating

This approach satisfies the simplified MVP requirements defined in the assessment.

6. Assumptions

The following assumptions were made during development

- All data is simulated.
- Users are assumed to be authenticated after selecting a role.
- GPS coordinates are mocked.
- Digital confirmations are generated locally.
- Ratings are stored locally.
- No payment processing is included.
- No real-time communication is implemented.

7. Out of Scope

The following features were intentionally excluded because they fall outside the MVP requirements:

- Real backend API
- Database integration
- JWT authentication
- Live GPS tracking
- Payment gateway integration
- AI-powered matchmaking
- SMS and Email notifications
- Mobile application
- Cloud deployment

8. Conclusion

The TAMP MVP successfully demonstrates the complete frontend workflow of the Truck Asset Matchmaking Platform using modern web technologies and mock data.

The application satisfies the primary business requirements by providing a responsive, role-based user interface that supports cargo posting, truck availability management, rule-based matching, digital confirmation, trip tracking, and administrative oversight. The modular architecture also provides a solid foundation for future backend integration and feature expansion.